Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 1

Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

Executable C code :

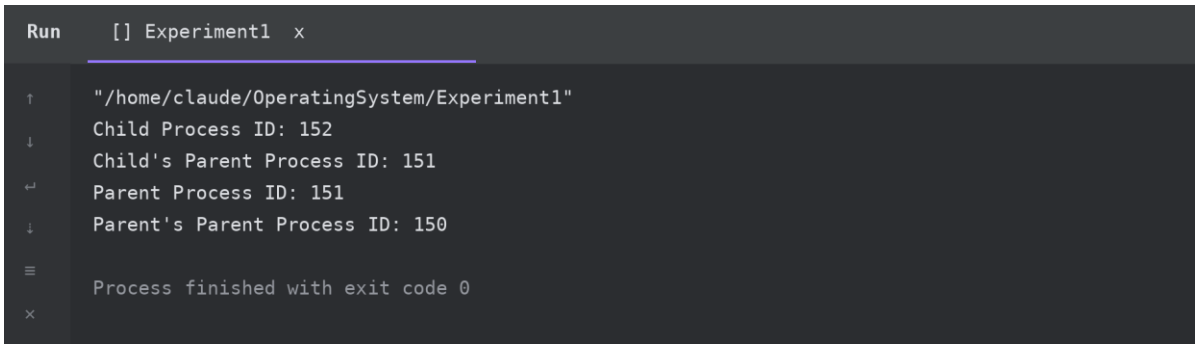
```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();   // create a new process

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        // child process
        printf("Child Process ID: %d\n", getpid());
        printf("Child's Parent Process ID: %d\n", getppid());
    }
    else
    {
        // parent process
        wait(NULL);   // wait for the child to finish
        printf("Parent Process ID: %d\n", getpid());
        printf("Parent's Parent Process ID: %d\n", getppid());
    }
    return 0;
}
```

Output Screenshot:



Experiment 2

Identify the system calls to copy the content of one file to another and illustrate the same using a C program.

Executable C code :

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fptr1, *fptr2;
    char filename[100];
    int c;

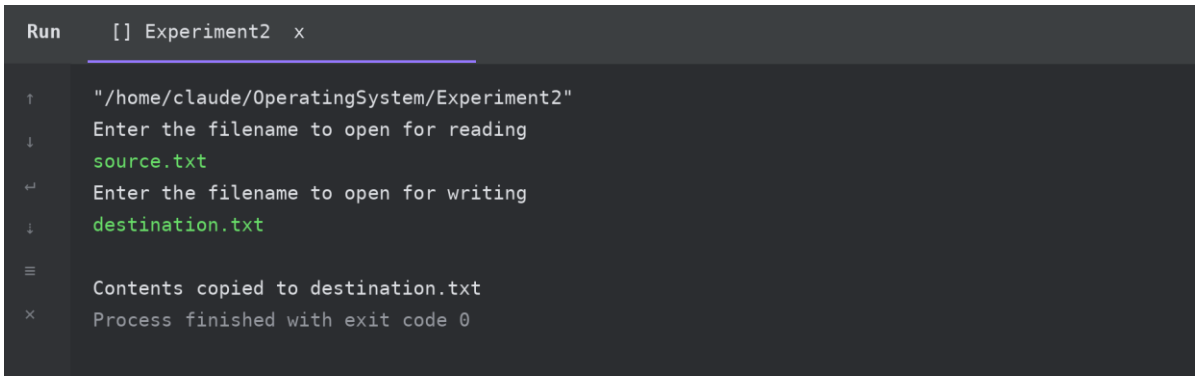
    printf("Enter the filename to open for reading \n");
    scanf("%s", filename);
    fptr1 = fopen(filename, "r");
    if (fptr1 == NULL)
    {
        printf("Cannot open file %s \n", filename);
        exit(0);
    }

    printf("Enter the filename to open for writing \n");
    scanf("%s", filename);
    fptr2 = fopen(filename, "w");
    if (fptr2 == NULL)
    {
        printf("Cannot open file %s \n", filename);
        exit(0);
    }

    c = fgetc(fptr1);
    while (c != EOF)
    {
        fputc(c, fptr2);
        c = fgetc(fptr1);
    }

    printf("\nContents copied to %s", filename);
    fclose(fptr1);
    fclose(fptr2);
    return 0;
}
```

Output Screenshot:



Experiment 3

Design a CPU scheduling program with C using First Come First Served technique with the following considerations.

a. All processes are activated at time 0.

b. Assume that no process waits on I/O devices.

Executable C code :

```c
#include <stdio.h>

int main() {
    int n, i, bt[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

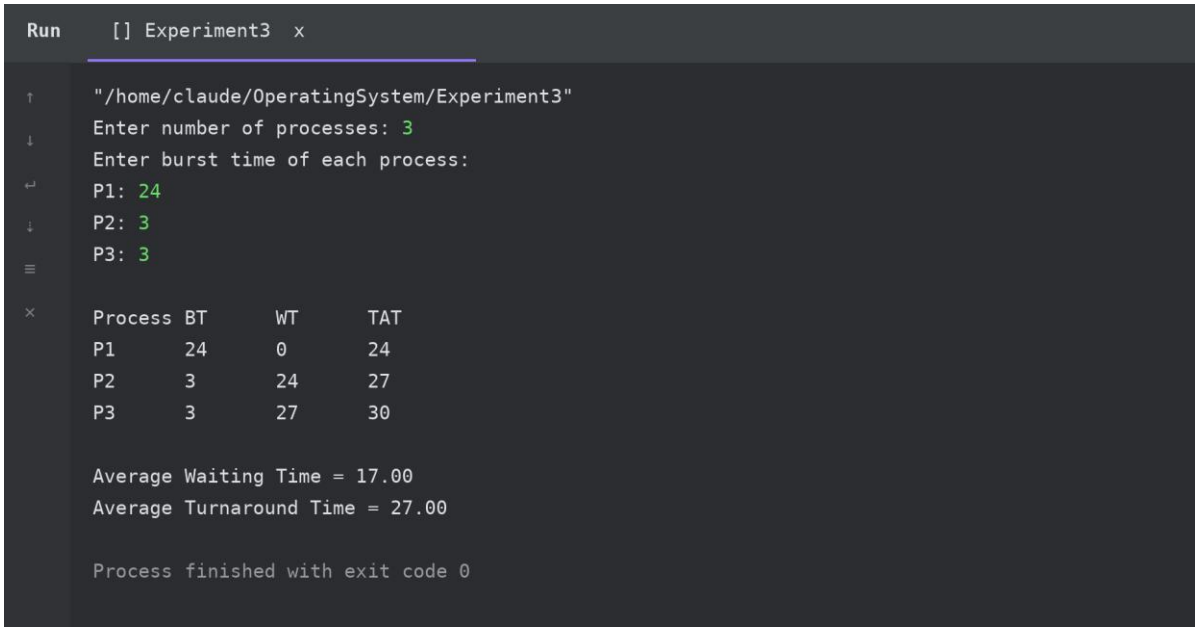
    printf("Enter burst time of each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
    printf("Average Turnaround Time = %.2f\n", avgtat / n);
    return 0;
}
```

Output Screenshot:



Experiment 4

Construct a scheduling program with C that selects the waiting process with the smallest execution time to execute next.

Executable C code :

```
#include <stdio.h>

int main() {
    int n, i, j, temp, p[10], bt[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time of each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        p[i] = i + 1;
    }

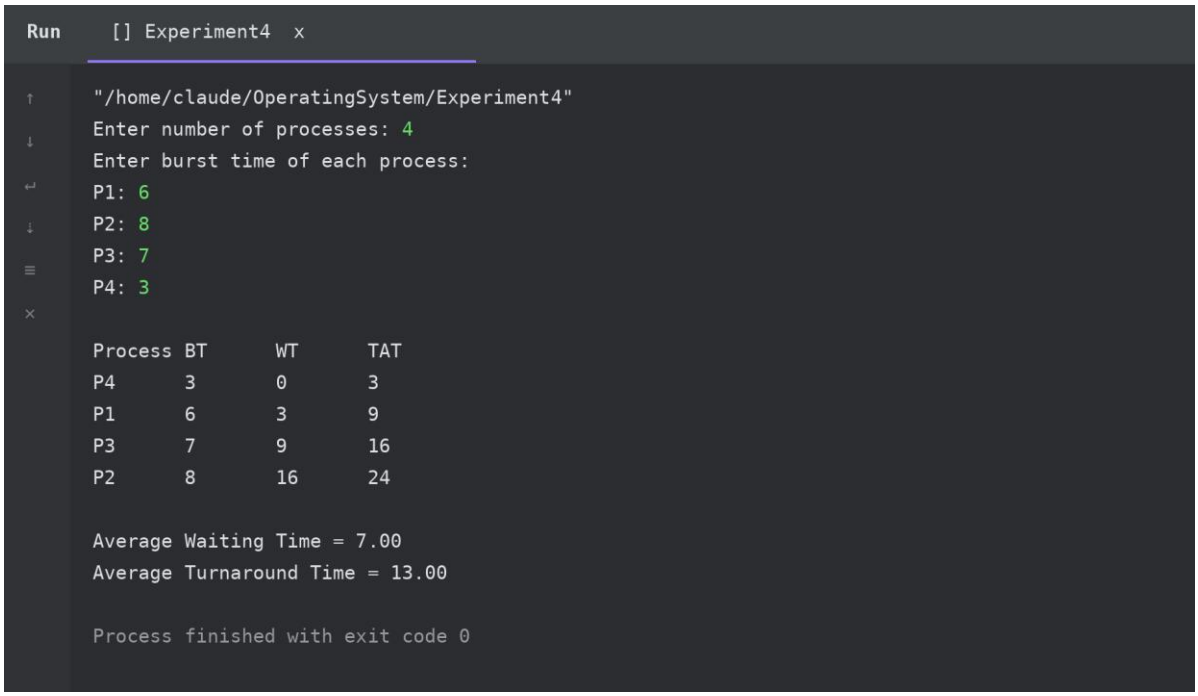
    // sort by burst time (smallest first)
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i];  p[i] = p[j];   p[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
    printf("Average Turnaround Time = %.2f\n", avgtat / n);
    return 0;
}
```

Output Screenshot:



Experiment 5

Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

Executable C code :

```
#include <stdio.h>

int main() {
    int n, i, j, temp, p[10], bt[10], pr[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority of each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d %d", &bt[i], &pr[i]);
        p[i] = i + 1;
    }

    // sort by priority (lower number = higher priority)
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (pr[i] > pr[j]) {
                temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i];  p[i] = p[j];   p[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
        printf("P%d\t%d\t\t%d\t%d\t%d\n", p[i], pr[i], bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
    printf("Average Turnaround Time = %.2f\n", avgtat / n);
    return 0;
}
```

Output Screenshot:

